



AGENT SKILLS · AULA 1

Por que usar *Skills?*

Como as Skills tornam agentes de IA mais confiáveis, previsíveis e fáceis de manter em produção — transformando improvisação em processo.

O problema

Anatomia

5 vantagens

Casos de uso



Agentes sem Skill: *o que dá errado?*

Sem instruções estruturadas, o agente **improvisa** — e improviso gera resultados inconsistentes.



Imprevisível

O mesmo prompt produz respostas diferentes a cada execução.



Retrabalho

Cada tarefa exige re-explicar contexto, formato e regras. Zero reuso.



Qualidade cai

Sem playbook, o agente toma atalhos e pula etapas.



Difícil auditar

Sem registro de como agir, impossível rastrear um comportamento errado.



Não escala

Cada dev configura o agente de um jeito. Sem padrão, vira caos.



Sem guardrails

Decisões prejudiciais por falta de regras e limites explícitos.

Skills existem para resolver exatamente isso — improvisação vira processo

O que é uma *Skill*?

Um arquivo Markdown (`SKILL.md`) com instruções estruturadas — um **playbook** — injetado no contexto do agente quando necessário. Ela não executa código: **ensina o modelo a pensar e agir** de forma consistente.



Arquivo Markdown

Só texto. Sem compilação, sem SDK, sem runtime. Qualquer editor funciona.



Injetada no contexto

O conteúdo entra no system prompt quando o agente identifica relevância.



Instrui o raciocínio

Etapas, regras, formatos de output e stop conditions.



Estrutura mínima — `SKILL.md`

```
---  
name: minha-skill  
description: O que ela faz  
version: 1.0.0  
---
```

```
## Instruções  
Quando o usuário pedir X:  
  1. Faça A  
  2. Faça B  
  3. Confirme o resultado
```

5 razões para usar *Skills*

01

Reusabilidade

Escreva uma vez, use em todos os agentes e projetos.

02

Consistência

Mesmo fluxo toda vez — previsível em produção.

03

Manutenção

Edite o `.md` e todos os agentes se atualizam.

04

Auditabilidade

Todo comportamento documentado em texto. Sem caixa-preta.

05

Escalabilidade

Equipes seguem o mesmo padrão. Onboarding em minutos.

Skills criam uma linguagem comum entre pessoas e agentes

Dentro de um *SKILL.md*

```
---
name: code-review
description: Revisa PRs por segurança
version: 2.1.0
bins: [git, rg]
---

## Gatilho
Quando pedirem "revise este PR".

## Fluxo
1. Cheque SQL injection e XSS
2. Verifique secrets expostos
3. Gere relatório por severidade

## Regras
- Confirme antes de aprovar
- Se achar secret, pare e avise
```

Frontmatter

Metadados YAML: nome, descrição, versão e dependências — usados para descobrir e filtrar a Skill.

Dependências

`bins` : binários no PATH · `env` : variáveis. A Skill só ativa se tudo existir.

Gatilho

Define **quando** acionar. Quanto mais específico, menor o risco de ativações indevidas.

Fluxo

Passos numerados = checklist determinístico. O agente executa na ordem, sem improvisar.

Regras / Guardrails

Stop conditions e limites explícitos evitam decisões perigosas.

Sem Skill *vs* Com Skill

SITUAÇÃO	😓 SEM SKILL	✅ COM SKILL
Novo agente	Dev re-explica tudo; começa do zero	Carregada automaticamente; já sabe o que fazer
Bug no comportamento	Ninguém sabe por quê; sem rastro	Lê o SKILL.md e acha o passo problemático
Novo membro na equipe	Horas de explicação	Lê o SKILL.md em 5 min e entende o fluxo
Atualizar comportamento	Altera N prompts em N lugares	Edita um único arquivo; todos atualizam
Produção	Resultado imprevisível; SLA difícil	Padronizado e auditável; fácil de monitorar

Um arquivo de texto vira a fonte única da verdade sobre como o agente age

Onde Skills *brilham*



Code Review

Padroniza checklist de segurança, formato do relatório e critérios de aprovação de PRs.



Gestão de e-mails

Define triagem, priorização, tom de resposta e quando escalar.



Relatórios semanais

Automatiza coleta de dados, formato e distribuição — igual toda semana.



Atendimento

Garante tom, políticas de reembolso e escalção correta.



Auditoria de segurança

Segue OWASP Top 10 em ordem, com severidade padronizada.



Planejamento de sprint

Conduz o ritual: backlog, estimativas, dependências e board.

💡 Antes de criar do zero, verifique marketplaces de Skills prontas para instalar e adaptar

Como escrever uma *boa Skill*

Expanda cada princípio.

Seja específico nos gatilhos



Descreva exatamente quando a Skill deve ser acionada. Gatilhos vagos causam falsos positivos e ativações indevidas.

Use listas numeradas



Fluxos em etapas são mais confiáveis que parágrafos — o agente segue um checklist melhor do que prosa.

Defina stop conditions



Sempre inclua "Se X acontecer, pare e pergunte ao usuário". Evita decisões autônomas arriscadas.

Mantenha curta e focada



Skills longas aumentam o custo de tokens e diluem o foco. Uma Skill = uma responsabilidade.

Teste em isolamento



Antes de produção, rode em ambiente controlado. Observe edge cases e refine as regras.

Skills • Tools • MCP • *Subagentes*

Cada peça tem um papel diferente no sistema de agentes. A próxima aula aprofunda cada uma.



Skills

Instruções em Markdown.
Ensinam o agente a pensar e seguir um fluxo.



O manual



Tools

Funções executáveis. O agente age no mundo (API, arquivo, BD).



As ferramentas



MCP

Protocolo de conexão. Liga o agente a serviços externos padronizados.



O adaptador



Subagentes

Agentes especializados. Delegam subtarefas complexas.



A equipe

Skill = como pensar • Tool = o que executar • MCP = como conectar • Subagente = quem trabalha em paralelo

Teste *rápido*

Sua equipe tem 5 agentes que revisam PRs, cada um com o checklist ligeiramente diferente. Como padronizar em um só lugar?

A Copiar o mesmo prompt nos 5 agentes

B Criar uma Skill code-review e usá-la nos 5

C Criar uma Tool que revisa PRs

D Um subagente para cada PR

Resposta: B. Uma **Skill** centraliza o playbook: escreve-se o checklist uma vez e os 5 agentes carregam o mesmo `SKILL.md` — editar o arquivo atualiza todos. Copiar prompts gera inconsistência; a tarefa aqui é **padronizar o raciocínio**, não executar código (Tool) nem paralelizar (subagente).



 PRÓXIMO PASSO

Crie sua primeira *Skill*

1 **Identifique uma tarefa repetitiva**
Algo que seu agente faz sempre da mesma forma

2 **Escreva um SKILL.md**
Frontmatter + fluxo numerado + regras/guardrails

3 **Teste, refine e distribua**
Valide edge cases e compartilhe com a equipe

Próxima aula → **Skill · Tool · MCP & Subagentes**: os quatro pilares em detalhe.